
UNIT IV- PL/SQL PROGRAMMING

4.6 Functions:

Advantages, Creating, Executing and Deleting a Function.

Prepared By
Mr. Mohammad Ali
Lecturer (Selection Grade)
M. H. Saboo Siddik Polytechnic, Mumbai

PL/SQL - Functions

- A PL/SQL function is same as a procedure except that it **returns** a value.
- Functions can accept **one, many, or no parameters**, but a function **must have a return clause** in the executable section of the function.
- The **data type** of the **return value must be declared** in the header of the function.
- A function is not a stand-alone executable in the way that a procedure is: It must be used in some context.
- A function has output that needs to be assigned to a variable, or it can be used in a SELECT statement.

Creating a Function - Syntax

- A function is created using the CREATE FUNCTION statement.
- The simplified syntax for the CREATE OR REPLACE FUNCTION statement is as follows:

```
CREATE OR REPLACE FUNCTION <function_name>
(<variable_name / parameter_name> IN <datatype>,
<variable_name / parameter_name > IN <datatype>,...)
RETURN <datatype> {IS/AS}
variable/constant declaration;
BEGIN
    -- PL/SQL subprogram body <function body>;
EXCEPTION
    -- Exception Handling block ;
END <function_name>;
```

Creating a Function - Syntax

Where,

- ***function-name*** specifies the name of the function.
- ***variable_name*** is the variable name for variable used in the function.
- ***CREATE*** is a keyword used for specifying the name of the function to be created.
- ***[OR REPLACE]*** option allows modifying an existing function.
- The ***AS*** keyword is used instead of the ***IS*** keyword for creating a standalone function.
- The optional parameter list contains name, mode and types of the parameters. ***IN*** represents that value will be passed from outside.
- ***BEGIN, EXCEPTION*** and ***END*** are keywords used to indicate different sections of the function being created.
- ***RETURN*** clause specifies that data type we are going to return from the function. The function must contain a return statement.
- ***PL/SQL subprogram body (function-body)*** contains the executable part.

Calling/Executing a Function

- To use a function, we will have to call that function to perform the defined task. When a program calls a function, program control is transferred to the called function.
- A called function performs defined task and when its return statement is executed or when its last end statement is reached, it returns program control back to the main program.
- To call a function we simply need to pass the required parameters along with function name and if function returns a value then we can store returned value.

Example: Creating and executing Function to add two numbers

```
CREATE OR REPLACE FUNCTION total(a IN number, b IN number) RETURN Number IS
c number;
BEGIN
    c := a+b;
    RETURN c;
END;
/
```

OUTPUT:

Function created.

Calling the Function through PL/SQL code :

```
set verify off
DECLARE
n1 number;
n2 number;
result number;
BEGIN
n1 := &number1;
n2 := &number2;
result := total(n1,n2);
dbms_output.put_line('Sum of two nos='||result);
END;
/
```

OUTPUT:

```
Enter value for number1: 10
Enter value for number2: 20
Sum of two nos=30
```

PL/SQL procedure successfully completed.

Calling the function directly by providing two parameters
on SQL prompt through dummy table DUAL:

```
SQL> select total(10,20) from dual;

TOTAL(10,20)
-----
          30
```

Example:

Creating and executing Function to count total no of Employees in EMP

```
CREATE OR REPLACE FUNCTION totalEmp  
RETURN number IS  
    total number(2) := 0;  
BEGIN  
    SELECT count(*) into total FROM emp;  
    RETURN total;  
END;  
/
```

When above code is executed using SQL prompt, it will produce the following result:

Function created.

Following program calls the function totalEmp from a PL/SQL block:

DECLARE

c number(2);

BEGIN

c := totalEmp();

dbms_output.put_line('Total no. of Employees : ' || c);

END;

/

- When the above code is executed at SQL prompt, it produces the following result:

Total no. of Employees: 14

PL/SQL procedure successfully completed.

Example: Creating and executing Function to return maximum of two numbers

```
DECLARE
    a number; b number; c number;
FUNCTION findMax(x IN number, y IN number)
RETURN number
IS
    z number;
BEGIN
    IF x > y THEN
        z:= x;
    ELSE z:= y;
    END IF;
    RETURN z;
END;
BEGIN
    a:= 50; b:= 65;
    c := findMax(a, b);
    dbms_output.put_line(' Maximum of ' ||a|| ' & ' ||b|| ' : ' || c);
END;
```

/

When the above code is executed at SQL prompt, it produces the following result:

```
SQL> DECLARE
  2  a number; b number; c number;
  3  FUNCTION findMax(x IN number, y IN number)
  4  RETURN number
  5  IS
  6  z number;
  7  BEGIN
  8  IF x > y THEN
  9  z:= x;
 10 ELSE Z:= y;
 11 END IF;
 12 RETURN z;
 13 END;
 14 BEGIN
 15 a:= 50; b:= 65;
 16 c := findMax(a, b);
 17 dbms_output.put_line(' Maximum of ' ||a|| ' & ' ||b|| ' : ' || c);
 18 END;
 19 /
```

Maximum of 50 & 65 : 65

PL/SQL procedure successfully completed.

SQL>

Deleting a Function

- Syntax:

DROP FUNCTION <function_name>

- Example:

```
SQL> DROP FUNCTION totalemp;  
Function dropped.  
SQL>
```